

Inigo Quilez :: articles :: voronoi - 2014

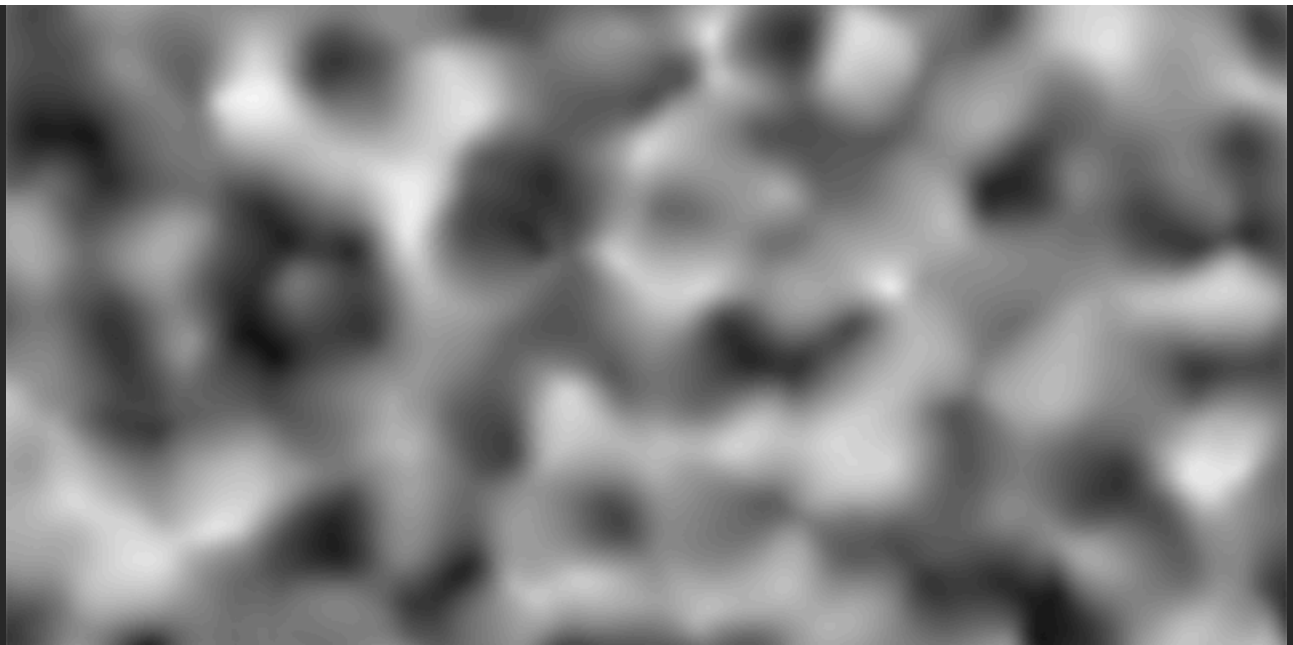


Intro

Two of the most common building blocks for procedural pattern generation are Noise, which have many variations (Perlin's being the first and most relevant), and Voronoi (also known as "celular") which also has different variations. For Voronoi, the most common of those variations is the one that splits the domain in a regular grid such that there's one feature point in each of the cells. That means that Voronoi patterns are based on a grid after all just like Noise, the difference being that while in Noise the feature originators are in the vertices of the grid (random values or random gradients), Voronoi has the feature generators jittered somewhere in the grid. That might be a first indicator that, perhaps, the two patterns are not that unrelated, at least from an implementation perspective?

Despite this similarity, the fact is that the way the grid is used in both patterns is different. Noise interpolates/averages random values (as in **value noise**) or gradients (as in **gradient noise**), while Voronoi computes the distance to the closest feature point. Now, smooth-bilinear interpolation and minimum evaluation are two very different operations, or... are they? Can they perhaps be combined in a more general metric? If that was so, then both Noise and Voronoi patterns could be seen as particular cases of a more general grid-based pattern generator?

This article is about a small effort to find such generalized pattern. Of course, the code implementing such generalization will never be as fast as implementations of the particular cases (rendering this article with no obvious immediate practical purpose), but at least it might open the window to a bigger picture understanding and perhaps, one day, new findings!



Voronoise - a combination of Voronoi, and Noise

The code

In order to generalize Voronoi and Noise, we must introduced two parameters: one to control the amount of jittering of the feature points, and one for controlling the metric. Let's call the grid control parameter **u**, and the metric controller **v**.

The grid parameter is pretty simple to design: **u=0** will simply use a Noise-like regular grid, and **u=1** will be the Voronoi-like jittered grid. So, the value of **u** can simply control the amount of jitter. Straightforward.

The **v** parameters will have to blend between a Noise-like bilinear interpolator of values, and a Voronoi-like min operator. The main difficulty here is that the min() operation is a non-continuous function. However, luckily enough for us, there are smooth alternatives such as the **Smooth Voronoi**. If we apply a power functions to the distance to each feature points in order to highlight the closest one over the rest, then we get a nice side effect: using a power of 1 gives all features the same relevance and therefore we get an equal interpolation of features, which is what we need for Noise-like patterns! So, something like this might do it:

```
float ww = pow( 1.0-smoothstep(0.0,1.414,sqrt(d)), 64.0 - 63.0*v );
```

However, a bit of experimentation proves that a better perceptually linear interpolation between the Noise-like and the Voronoi-like pattern can be achieved by rising **v** to some power:

```
float ww = pow( 1.0-smoothstep(0.0,1.414,sqrt(d)), 1.0 + 63.0*pow(1.0-v,4.0) );
```

So, it seems that after all it's not so difficult to generalize Noise and Voronoi. Therefore, assuming one has a way to generate random values deterministically as a function of the grid cell id (which you are already doing both in your favorite Voronoi and Noise implementations), which we could call

```
vec3 hash3( in vec2 p )
```

then the code for our new generalized super pattern could be like this:

```
float noise( in vec2 x, float u, float v )
{
    vec2 p = floor(x);
    vec2 f = fract(x);

    float k = 1.0 + 63.0*pow(1.0-v,4.0);
    float va = 0.0;
    float wt = 0.0;
    for( int j=-2; j<=2; j++ )
    for( int i=-2; i<=2; i++ )
    {
        vec2 g = vec2( float(i), float(j) );
        vec3 o = hash3( p + g )*vec3(u,u,1.0);
        vec2 r = g - f + o.xy;
        float d = dot(r,r);
        float w = pow( 1.0-smoothstep(0.0,1.414,sqrt(d)), k );
        va += w*o.z;
        wt += w;
    }

    return va/wt;
}
```

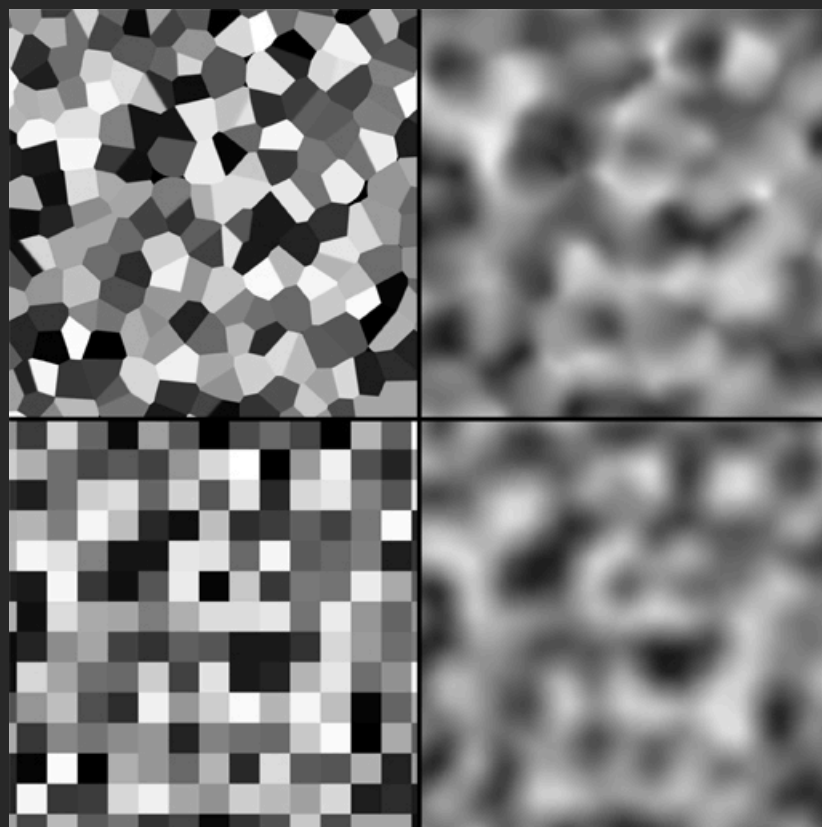
The implementation is very similar to the **regular Voronoi pattern**, the difference being that we now have the weighted average of distance computations happening (the accumulation happens in **va** and the counting for later normalization is in **wt**).

Results

The results of the generalization are rather interesting. Of course, we have generalized Noise and Voronoi. Indeed, noise happens when **u=0, v=1**, ie, regular grid and interpolation of feature distances. Voronoi happens when **u=1, v=0**, ie, when the grid is jittered and the metric is the minimum distance.

However there's two side effects. The first happens when $u=0$, $v=0$, which gives a minimum distance to a non jittered grid of features. This basically gives a pattern with a constant value per grid cell, or what normally is called "cell noise".

The second side effect happens for $u=1$, $v=1$, which generates a pattern that has an interpolated value of distances to features in a jittered grid. It's a combination of Voronoi and Noise, or as I am naming it, **Voronoise** (top right in the image). This pattern can be useful for regular procedural generation where grid artifacts are visible, because the jittering certainly hides the underlying grid structure of Noise.



Bottom Left: $u=0$, $v=0$: Cell Noise

Bottom Right: $u=0$, $v=1$: Noise

Top Left: $u=1$, $v=0$: Voronoi

Top Right: $u=1$, $v=1$: Voronoise

A realtime interactive implementation of the code above can be found here (click in the title to navigate to the source code, or simply move the mouse along the image to vary the u and v parameters).



inigo quilez - learning computer graphics since 1994